

INTERMEDIATE SQL

Foreign key

A foreign key is a column (or group of columns) in one table that refers to the primary key of another table.

It is used to create a relationship between tables and maintain referential integrity.

Example

We have two tables.

1. Students table

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

student_id	name
1	John
2	Mary

2. Classes table

```
CREATE TABLE Classes (  
    class_id INT PRIMARY KEY,  
    class_name VARCHAR(50),  
    student_id INT,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id)  
);
```

class_id	class_name	student_id
101	Math	1
102	Science	2

Here:

Students

student_id ← Primary Key

↑

|

| Foreign Key

|

Classes

student_id

So Classes.student_id refers to Students.student_id.

For example, this is valid:

```
INSERT INTO Classes
VALUES (103, 'English', 1);
```

Because student 1 exists.

But this is not valid:

```
INSERT INTO Classes
VALUES (104, 'English', 99);
```

Because student 99 doesn't exist.

Basic Syntax

```
ALTER TABLE `new_schema`.`orders`  
  ADD CONSTRAINT `orders_user_id_key`  
  FOREIGN KEY (`user_id`)  
  REFERENCES `new_schema`.`users` (`id`);
```

Transactions

A transaction is a group of SQL statements that are treated as one unit of work.

example:

transferring ₹100 from Account A to Account B.

You want both operations to happen, or neither should happen.

Basic Syntax

START TRANSACTION;

UPDATE accounts

SET balance = balance - 100

WHERE account_id = 1;

UPDATE accounts

SET balance = balance + 100

WHERE account_id = 2;

COMMIT;

Steps

- START TRANSACTION → starts the transaction
- UPDATE → performs the changes
- COMMIT → permanently saves the change

ACID

ACID describes four important properties of database transactions.

A = Atomicity C = Consistency I = Isolation D = Durability

Example

a simple **bank transfer** example.

Suppose:

Account A = ₹1000

Account B = ₹500

We transfer ₹200 from A to B

1. Atomicity — "All or Nothing"

Both operations must happen, or neither happens.

START TRANSACTION;

UPDATE accounts

SET balance = balance - 200

```
WHERE account_id = 1;
```

```
UPDATE accounts
```

```
SET balance = balance + 200
```

```
WHERE account_id = 2;
```

```
COMMIT;
```

If the second operation fails:

```
ROLLBACK;
```

The first operation is also undone.

Atomicity = Everything succeeds or everything is cancelled.

2. Consistency — "Valid Data"

The database must remain in a valid state before and after the transaction.

For example, if an account cannot have a negative balance, the transaction should not violate that rule.

Before transaction:

A = ₹1000

B = ₹500

After transaction:

A = ₹800

B = ₹700

The database rules are still satisfied.

Consistency = Data remains correct according to the rules.

3. Isolation — "Transactions Don't Interfere"

If two users perform transactions at the same time, one transaction should not incorrectly interfere with another.

For example:

Transaction 1 → Transfer ₹200

Transaction 2 → Transfer ₹100

MySQL's transaction isolation mechanisms control what each transaction can see while other transactions are running.

Isolation = Concurrent transactions are kept appropriately separate.

4. Durability — "Once Saved, It Stays"

After:

COMMIT;

the changes are permanently saved.

Even if the database server crashes afterward, MySQL is designed to recover committed changes.

Durability = Committed data is not lost.